PRÁCTICA DE LABORATORIO: GESTIÓN DE FLUJOS DE ENTRADA (BUFFERING) Y CLASES ENVOLTORIO (WRAPPER CLASSES) EN JAVA

Programación Orientada a Objetos

1. OBJETIVOS DE APRENDIZAJE

- Comprender la arquitectura de bajo nivel del buffer de entrada en Java (System.in) y el comportamiento de la clase Scanner frente a los caracteres delimitadores (\n).
- Resolver de manera técnica la omisión o bloqueo en la lectura de cadenas de texto provocada por saltos de línea residuales en la memoria intermedia.
- Aplicar clases envoltorio (*Wrapper classes*: Integer, Double, Character, Boolean) para el tratamiento de tipos primitivos como objetos dentro de colecciones dinámicas de la API de Java (java.util).
- Utilizar métodos estáticos de análisis y conversión (*parsing*) que proveen las clases envoltorio para garantizar una lectura robusta de datos en consola.

2. FUNDAMENTOS TEÓRICOS Y ARQUITECTURA

Concepto	Descripción Técnica	Impacto en la Ejecución
Buffer de Teclado	Área temporal de memoria donde se almacenan las pulsaciones antes de ser consumidas por la aplicación.	Cada tecla pulsada y la confirmación ENTER viajan como códigos ASCII al flujo de entrada.
La Trampa del \n	Métodos como nextInt() o nextDouble() solo extraen los caracteres que componen el token numérico, dejando huérfano el salto de línea \n.	Al invocar posteriormente nextLine(), este detecta de inmediato el \n restante y retorna una cadena vacía sin pausar la consola para el usuario.
Purga de Flujo	Invocación explícita de nextLine() sin asignación tras una lectura de token numérico.	Vierte y descarta el delimitador residual, restableciendo el puntero del buffer para capturar nuevas cadenas.
Wrapper Classes	Objetos del paquete java.lang que encapsulan tipos primitivos (int → Integer, double → Double, etc.).	Permiten que tipos primitivos sean tratados como subclases de Object, requisito indispensable para integrarlos en colecciones genéricas como List y ArrayList.

Una institución financiera les encarga construir un módulo de consola para registrar operaciones bancarias en ventanilla en tiempo real. La interfaz parece elemental: el cajero debe ingresar el identificador de la cuenta, el nombre completo del cuentahabiente, el monto a transferir y si la operación fue autorizada o no.

Sin embargo, al ejecutar la primera prueba en producción, el sistema presenta dos anomalías críticas:

1. **La pérdida de entrada por salto de línea:** En cuanto el cajero teclea el ID numérico y presiona `ENTER`, la aplicación salta de forma automática el campo del nombre del cuentahabiente dejándolo vacío, pasando directo a solicitar el monto.
2. **Incompatibilidad de tipos y rigidez de datos:** El motor contable central exige almacenar todos los registros dentro de colecciones dinámicas de la API de Java (`List<E>`), las cuales operan exclusivamente bajo la jerarquía de `Object` y rechazan tipos primitivos directos. Además, recibir valores numéricos o estados booleanos desde una interfaz de texto cruda sin métodos utilitarios de conversión expone a la aplicación a bloqueos de formato regional (comas vs. puntos decimales) o entradas inconsistentes de caracteres.

PASO 1: MODELADO DE DATOS CON WRAPPER CLASSES

Implementa en tu entorno de desarrollo (IDE) la clase de entidad **Transaccion** dentro del paquete laboratorio.banco. Esta clase define los atributos requeridos utilizando clases envoltorio en lugar de tipos primitivos directos.

Código Fuente de Referencia:

```
package laboratorio.banco;

/**
 * Representa una transacción bancaria utilizando clases envoltorio (Wrapper classes)
 * para asegurar compatibilidad con colecciones basadas en Object.
 */
public class Transaccion {
    // Atributos definidos mediante Wrapper Classes
    private Integer id;
    private String titular;
    private Double monto;
    private Boolean esAprobada;

    // Constructor completo
    public Transaccion(Integer id, String titular, Double monto, Boolean esAprobada) {
        this.id = id;
        this.titular = titular;
        this.monto = monto;
        this.esAprobada = esAprobada;
    }

    // Representación formateada en consola
```

```
@Override
public String toString() {
    return String.format("TX #%04d | Titular: %-18s | Monto: $%-10.2f | Estado: %s",
            this.id,
            this.titular,
            this.monto,
            this.esAprobada ? "APROBADA" : "RECHAZADA");
}
```

Análisis Técnico del Modelo:

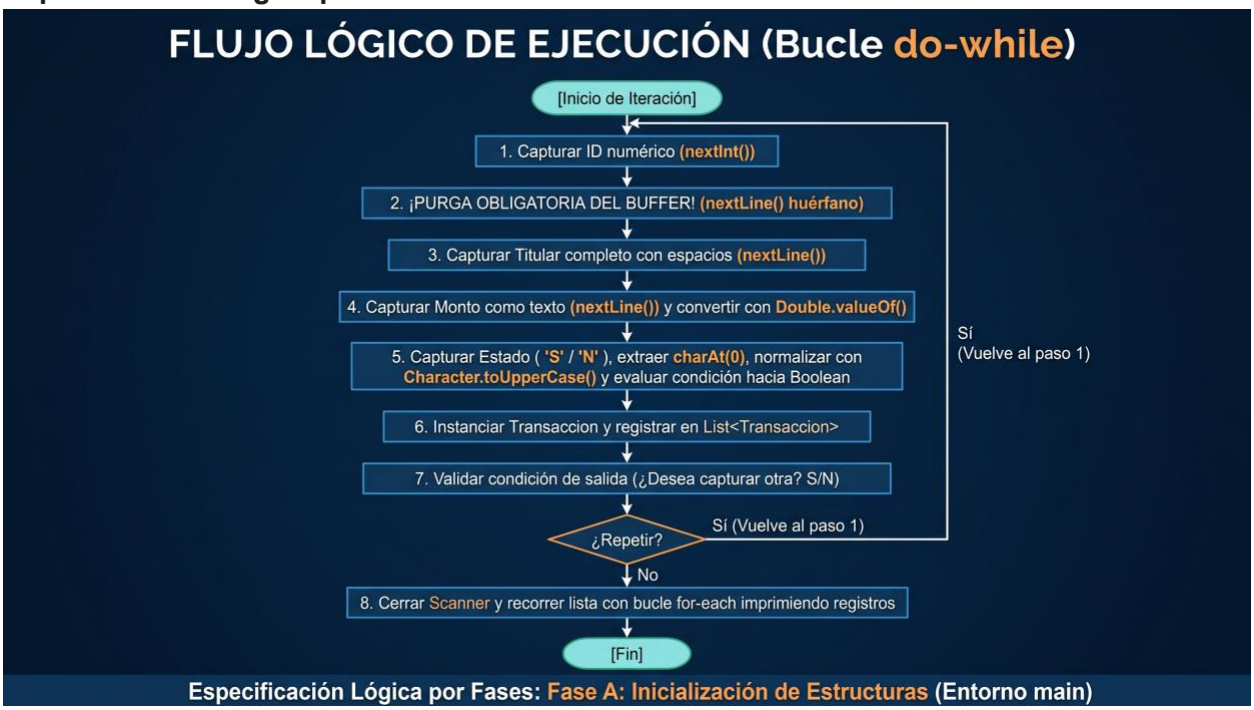
1. **Atributos de Objeto:** Se eligen Integer, Double y Boolean para permitir que los datos puedan ser manejados dentro de la jerarquía polimórfica de java.lang.Object.

2. **Método** toString()**:** Aplica especificadores de formato (%04d, %-18s, %-10.2f) para estandarizar la salida tabular en consola y traduce el booleano a una etiqueta de negocio entendible para el usuario final.

PASO 2: CONSTRUCCIÓN DEL PROCESADOR Y CONTROL DEL BUFFER

ATENCIÓN: Para esta sección no se proporciona código de solución. Deberás crear la clase GestorTransacciones implementando cada una de las fases y reglas lógicas que se detallan a continuación.

Especificación Lógica por Fases:



***Fase A: Inicialización de Estructuras (Entorno** main)*

1. Crea la clase GestorTransacciones con su método public static void main(String[] args).
2. Instancia un objeto de tipo Scanner vinculado a la entrada estándar System.in.
3. Declara e inicializa una lista dinámica de tipo List<Transaccion> utilizando la implementación ArrayList<Transaccion>. *(Nota: Las colecciones no soportan primitivos directos)*.
4. Declara una variable de control de tipo String para gobernar el bucle interactivo de captura.

Fase B: Bucle de Captura Iterativa

Implementa un bucle do-while que ejecute las siguientes acciones en cada iteración:

- **Lectura del Identificador (Numérico):**
  - Despliega un mensaje solicitando el ID de la transacción.
  - Lee el valor mediante el método nextInt() del escáner.
- **Desalojo Técnico del Buffer (Paso Crítico):**
  - *Explicación:* nextInt() leyó los dígitos numéricos, pero no extrajo el salto de línea \n enviado al presionar la tecla ENTER.
  - *Acción:* Invoca inmediatamente scanner.nextLine() de forma solitaria, sin guardar su valor en ninguna variable. Esta llamada absorbe y descarta el \n, dejando el buffer completamente vacío.
- **Lectura del Nombre Completo (Texto con espacios):**
  - Solicita el nombre completo del titular de la cuenta.

    - Captura la cadena completa utilizando scanner.nextLine(). *(Comprobarás que el programa se detiene a esperar el teclado de forma correcta gracias a la purga previa)*.
- **Parseo Numérico mediante Wrapper:**
    - Solicita el importe a transferir.
    - Lee el dato como una línea de texto mediante scanner.nextLine().
    - Convierte dicha cadena de texto a un objeto de tipo Double utilizando el método utilitario de la clase envoltorio: Double.valueOf(...).
- **Normalización de Carácter y Asignación Booleana:**
    - Pregunta al usuario si la transacción ha sido autorizada solicitando un valor (S/N).
    - Lee la entrada del usuario como cadena, extrae el primer carácter con **.charAt(0)** y conviértelo a mayúscula utilizando el método estático de la clase envoltorio: **Character.toUpperCase(...)**.
    - Evalúa si dicho carácter es igual a 'S'. Asigna el resultado de esta expresión de igualdad lógica directamente a una variable de tipo envoltorio Boolean.
- **Persistencia en Memoria y Control de Bucle:**
    - Crea una nueva instancia de Transaccion enviando las variables obtenidas al constructor y agrégala al ArrayList.
    - Pregunta al usuario: ¿Desea registrar otra transacción? (S/N):.
    - Lee la respuesta y actualiza la condición de salida del ciclo do-while.

Fase C: Liberación de Recursos y Despliegue de Resultados

1. Una vez finalizado el bucle, cierra formalmente el objeto Scanner invocando su método .close() para prevenir fugas de descriptores de entrada en el sistema operativo.
2. Imprime un encabezado en consola y recorre la colección de transacciones utilizando un bucle **for-each (for (Transaccion t : lista)).**
3. Imprime cada elemento por consola para comprobar el formateo generado por el método toString().

Salida en pantalla esperada:

```
============================================================
       SISTEMA DE GESTIÓN DE TRANSACCIONES BANCARIAS
============================================================

--- Captura de Transacción ---
Ingrese ID de transacción: 101
Ingrese nombre completo del titular: María José Hernández
Ingrese el monto a transferir: 15450.75
¿Transacción autorizada? (S/N): s

¿Desea registrar otra transacción? (S/N): S

--- Captura de Transacción ---
Ingrese ID de transacción: 102
Ingrese nombre completo del titular: Carlos Eduardo Morales Peña
Ingrese el monto a transferir: 820.50
¿Transacción autorizada? (S/N): n

¿Desea registrar otra transacción? (S/N): S

--- Captura de Transacción ---
Ingrese ID de transacción: 103
Ingrese nombre completo del titular: Ángel Aguilar García
Ingrese el monto a transferir: 32900.00
¿Transacción autorizada? (S/N): S

¿Desea registrar otra transacción? (S/N): n

Cerrando lector de entrada y generando reporte...

================================================================================
                              HISTORIAL DE OPERACIONES
================================================================================
TX #0101 | Titular: María José Hernández   | Monto: $15450.75   | Estado: APROBADA
TX #0102 | Titular: Carlos Eduardo Morales | Monto: $820.50     | Estado: RECHAZADA
TX #0103 | Titular: Ángel Aguilar García   | Monto: $32900.00   | Estado: APROBADA
================================================================================
Total de registros procesados: 3
Programa finalizado con éxito.
```

RÚBRICA ANALÍTICA DE EVALUACIÓN

Criterio Técnico	Excelente (100%)	Aceptable (70%)	Insuficiente (0% - 40%)
Gestión y Purga de Buffer (35%)	Identifica y limpia el salto de línea \n mediante nextLine() solitario tras nextInt(). No se presentan omisiones de captura.	Limpia el buffer pero con instrucciones redundantes o desordenadas en el flujo.	Omite la limpieza del buffer; la consola se salta la lectura del nombre del titular.
Uso de Wrapper Classes (30%)	Aplica correctamente Integer, Double, Character y Boolean en modelado, parsing de cadenas y normalización de caracteres.	Utiliza wrappers en el modelo pero recurre a primitivos sin métodos de utilidad en la captura de datos.	No emplea wrapper classes o desconoce los métodos estáticos de conversión (valueOf, toUpperCase).
Persistencia en Colecciones (20%)	Declara y gestiona adecuadamente la interfaz List implementada con ArrayList parametrizada con objetos.	Emplea la lista pero con advertencias de tipos crudos (*raw types*) o variables fuera de alcance.	Intenta usar arreglos primitivos estáticos o no almacena los objetos creados.
Presentación y Formato (15%)	Formato de salida tabular alineado en consola, código limpio, identado y cierre correcto de recursos (scanner.close()).	Salida funcional pero desalineada; no cierra los flujos de lectura de forma explícita.	El programa no compila o presenta excepciones en tiempo de ejecución no resueltas.